

BUILDING INTELLIGENT, EVENT-DRIVEN ARCHITECTURES WITH FABRIC REAL-TIME INTELLIGENCE

Fabric IQ

Grounding real-time intelligence in enterprise context

Retail cold-chain monitoring, end to end

Brian Bønke

Founder & MVP, ProBI

Fabric IQ half of the day · 4 hours · 6 modules

Today's scenario, in one line

A retail chain with **stores**, **customers**, and **freezers** full of frozen goods.

- Freezers stream live temperature telemetry, second by second
- A raw reading like **-9°C** is meaningless on its own
- Fabric IQ adds the business context that turns a number into a decision



We spend the next four hours building this — live, in your own workspace.

Where this fits in the day

First half

Real-Time Intelligence

Eventstream, Eventhouse, Activator

Streaming freezer telemetry into the workspace

This half

Fabric IQ

The semantic and agent layer *on top of* RTI

Same retail cold-chain data, same workspace



We're extending, not restarting.

The four-hour build, top to bottom

Module	Topic	Theory / Lab
09	Fabric IQ architecture & why context matters	10 / 15 min
10	Telemetry + semantic context, grounding signals	15 / 25 min
11	Build ColdChainOntology — highest-value module	15 / 30 min
12	Agents on top: ColdChainDataAgent, ColdChainOperationsAgent	15 / 20 min
13	Prompting, trust, traceability — built-in buffer	10 / 5 min
14	Wrap-up & resources	10 min

Light RTI recap

You already played around with these items before lunch:

- **Eventstream** — Live data from sources, handled easily with simple transformations
- **Eventhouse** — Stores the streaming data for easy querying and analytics using the KQL language
- **Activator** — rule-based alerting on raw thresholds



Fabric IQ doesn't replace any of this — it sits on top of it.

	Name	Status	Type
	00_LoadReferenceData		Notebook
	ColdChainEventhouse		Eventhouse
	 ColdChainKQLDB		KQL Database
	ColdChainLakehouse		Lakehouse
	 ColdChainLakehouse		SQL analytic...
	FreezerTelemetryEventstream		Eventstream

The Fabric IQ workspace after provisioning

What you'll have when this module ends

Workspace

Fabric IQ

On a P1 capacity

Lakehouse

ColdChainLakehouse

Customers · Stores · Freezers

Eventhouse

ColdChainEventhouse

KQL DB ColdChainKQLDB

Eventstream

FreezerTelemetryEventstream

created and ready for the generator

Housekeeping

- Two breaks: after Module 10, and after Module 12
- **Module 13 is a built-in buffer** — if Modules 11/12 run long, we compress 05, not 03/04
- Modules 11 and 12 uses **UI** — if something misbehaves live, we blaim Microsoft for giving us new features :)
- Ask questions as we go — this is hands-on, not a lecture

HANDS-ON LAB

Lab 08 — Set up & verify

35 minutes

- **Part A:** install the Fabric CLI, sign in, and run `provision_fabric_iq.py` live
- **Part B:** everyone confirms the Fabric IQ workspace and items match the checklist



LAB GUIDE

aka.bi/lab8

MODULE 09 • TOPICS 1–2

Fabric IQ architecture & why context matters

10 min theory • 15 min lab

BY THE END OF THIS MODULE YOU CAN

- Describe where Fabric IQ sits relative to Lakehouse, Eventhouse, and Power BI
- Explain, in one sentence, why raw events need business context to be actionable
- Recognize the three architectural layers we'll build through today

TOPIC 1

What is Fabric IQ?

- Microsoft's **ontology / semantic / agent layer** on top of Fabric
- It doesn't store a second copy of your data — it **overlays meaning** on data that already lives in OneLake
- Built for one purpose: make operational data **understandable and actionable** by both people and AI agents



Think of it as the layer that turns "a table of numbers" into "a Freezer that belongs to a Store."

TOPIC 1

The architecture, three layers

03 Agent layer

Data Agent · Operations Agent · Activator Ontology Rules — reason and act over the semantic layer (*Module 12*)

02 Semantic layer

The **ontology** — ColdChainOntology: entities, properties, relationships, bound to the data layer with *no copying* (*Module 11*)

01 Data layer

Lakehouse ColdChainLakehouse · Eventhouse ColdChainEventhouse — what RTI already built

TOPIC 1

How the layers connect

Static context

Customer records, store
master data, freezer
specs

→ **Lakehouse**

Live signals

Freezer temperature,
door state

→ **Eventstream** →
Eventhouse

Ontology

Binds both, without
moving either

Agents

Query and act through
the ontology — never
against raw tables
directly

This is the same layering pattern as Microsoft's Digital Twin Builder tutorials — we're following a tested template, not inventing one.

TOPIC 2

Why context matters for event interpretation

- A telemetry event is just **a number with a timestamp**
- The same number can mean *nothing* or *everything*, depending on what it's attached to
- Without context, every downstream system — dashboards, alerts, agents — has to *guess* at meaning
- With context, the same event becomes a **traceable, explainable business fact**

What does -9°C mean?

Without context

-9°C

source: sensor-4471

...is that bad?

With context

-9°C

- Entity: **Freezer FZ-118**
- Normal range: **-18°C to -22°C**
- Store: **Copenhagen Nørrebro**

⚠ 9°C above spec — door open or compressor fault

SAME NUMBER

One version tells you nothing.

**The other tells you exactly what to do
and where to go.**

TOPIC 2

The same reading, a different entity

-9°C at a freezer

Spec: -18°C to -22°C

Alarming — act now

-9°C at a loading-dock cold room

Spec: -5°C to -10°C

Perfectly normal

- The raw event is identical — **only the entity context changes the required action**
- This is exactly what an ontology encodes: which entity, what's normal for it, what it relates to

TOPIC 2

Where we're headed

Module 10

Get the static and live data ready for grounding

Module 11

Build ColdChainOntology so "-9°C" always resolves to the right entity and meaning

Module 12

Put agents on top that reason in business terms, not raw thresholds

HANDS-ON LAB

Lab 09 — Architecture & context walkthrough

15 minutes

- Tour the Fabric IQ workspace and identify each architectural layer
- Inspect the raw telemetry stream and discuss what context is missing



LAB GUIDE

aka.bi/lab9

MODULE 10 • TOPICS 3–4

Telemetry & grounding

15 min theory • 25 min lab

BY THE END OF THIS MODULE YOU CAN

- Explain how static contextual data and live telemetry are prepared to meet in Fabric IQ
- Describe the update-policy vs. materialized-view distinction at a conceptual level
- Identify the keys that let a freezer's live signal and static record be linked later

TOPIC 3

Combining real-time telemetry with semantic context

- RTI already gives us a live stream: FreezerTelemetryEventstream → ColdChainEventhouse
- Fabric IQ's job is to make that stream **meaningful**, not just fast

Live signal

Temperature, door state, arriving every few seconds

Semantic context

Which freezer, which store, which customer base it serves

Neither ingredient alone is enough — this module gets both into the right shape.

TOPIC 3

The Digital Twin Builder pattern

Microsoft's official RTI tutorial establishes the template we're following:

1 • Upload

Static contextual data
(customers, stores,
freezers) as CSVs

2 • Land

Into a **Lakehouse**
(ColdChainLakehouse)
as governed tables

3 • Project

Into the Eventhouse so
it can be joined against
live telemetry

4 • Bind

The **ontology** (Module
11) binds the two
together for querying

Adapted from: Digital Twin Builder RTI tutorial, parts 1–2.

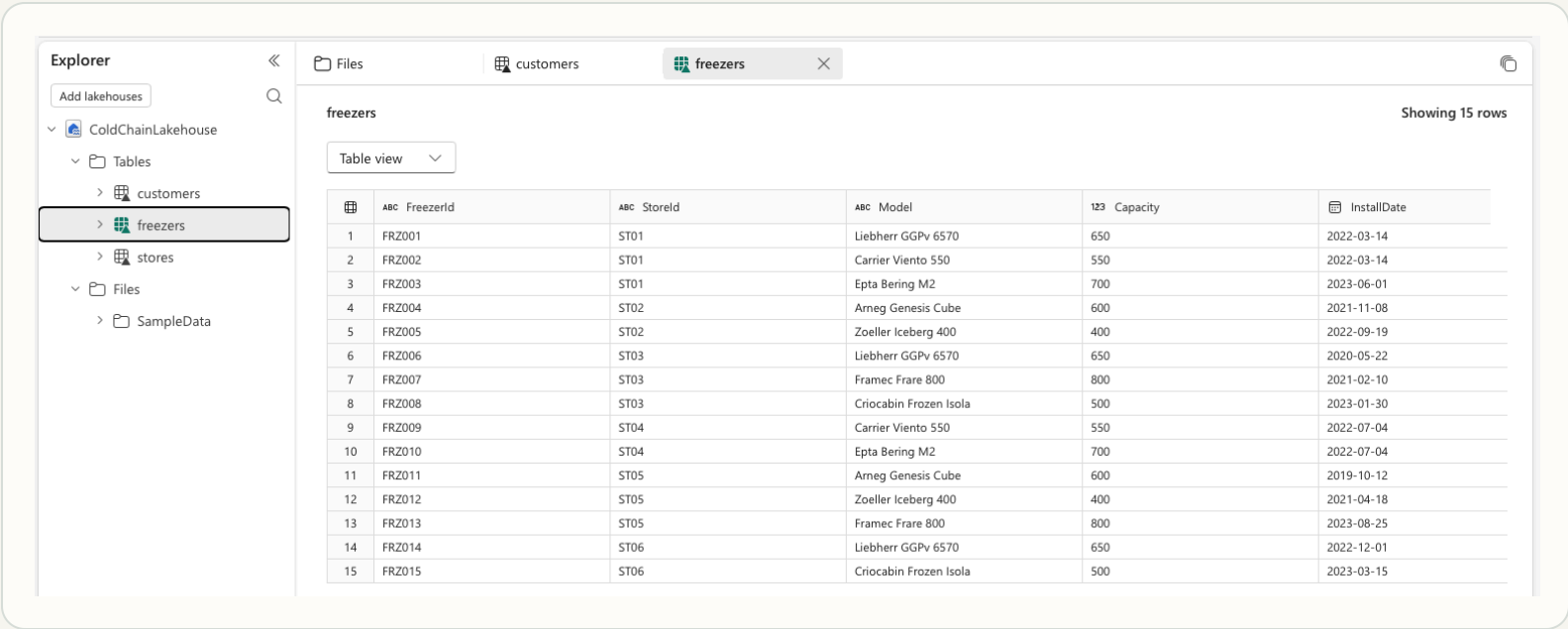
TOPIC 3

Where our reference data lives

ColdChainLakehouse

Table	Contents
Customers	Customer records
Stores	Store master data
Freezers	Model, Capacity, InstallDate — nameplate specs

This is **static context** — it changes rarely, if ever, during the day. The "what should be true" half of grounding.



ABC	FreezerId	StoreId	Model	Capacity	InstallDate
1	FRZ001	ST01	Liebherr GGPv 6570	650	2022-03-14
2	FRZ002	ST01	Carrier Viento 550	550	2022-03-14
3	FRZ003	ST01	Epta Bering M2	700	2023-06-01
4	FRZ004	ST02	Arneg Genesis Cube	600	2021-11-08
5	FRZ005	ST02	Zoeller Iceberg 400	400	2022-09-19
6	FRZ006	ST03	Liebherr GGPv 6570	650	2020-05-22
7	FRZ007	ST03	Framec Frare 800	800	2021-02-10
8	FRZ008	ST03	Criocabin Frozen Isola	500	2023-01-30
9	FRZ009	ST04	Carrier Viento 550	550	2022-07-04
10	FRZ010	ST04	Epta Bering M2	700	2022-07-04
11	FRZ011	ST05	Arneg Genesis Cube	600	2019-10-12
12	FRZ012	ST05	Zoeller Iceberg 400	400	2021-04-18
13	FRZ013	ST05	Framec Frare 800	800	2023-08-25
14	FRZ014	ST06	Liebherr GGPv 6570	650	2022-12-01
15	FRZ015	ST06	Criocabin Frozen Isola	500	2023-03-15

Freezers table in ColdChainLakehouse

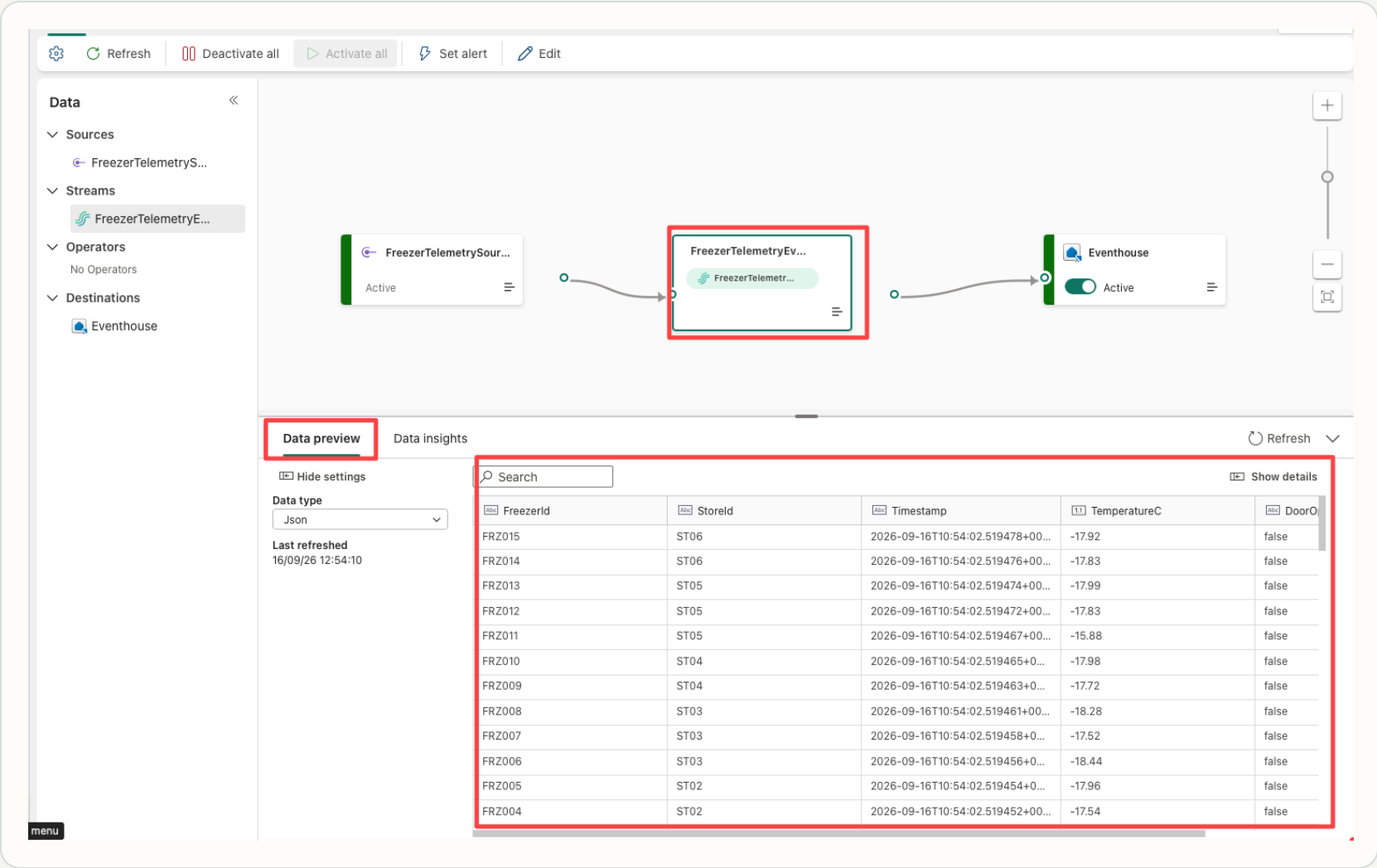
TOPIC 3

Where our live signal lives

CoIdChainEventhouse → CoIdChainKQLDB

Table	Contents
FreezerTelemetryRaw	Every event exactly as ingested
FreezerTelemetryEnriched	Cleaned/derived — TemperatureC, DoorOpen

This is **live signal** — it changes every few seconds. The "what is actually happening" half of grounding.



Live view of FreezerTelemetryEventstream

TOPIC 4

Grounding signals with enterprise meaning

- "Grounding" = attaching a live signal to the business entity it's actually about
- Before an ontology can bind anything, both sides need **consistent keys**
- A FreezerID in FreezerTelemetryEnriched must match a FreezerID in the Lakehouse Freezers table
- Get the keys wrong here, and no amount of ontology design later will fix it



This module's real deliverable: **clean, joinable data on both sides.**

TOPIC 4

Update policy vs. materialized view

Update policy

Runs on **ingest**, per event

Transforms Raw → Enriched automatically as data arrives

Good for: cleaning, reshaping, deriving fields continuously

Materialized view

Maintains a **pre-aggregated, always-fresh summary**

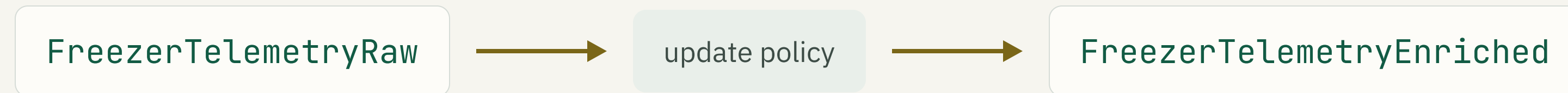
Good for: "current state per freezer," rollups, dashboards

Avoids re-scanning the whole raw table every query

Adapted from: Kusto update-policy docs, materialized-view use cases.

TOPIC 4

Applying that to our pipeline

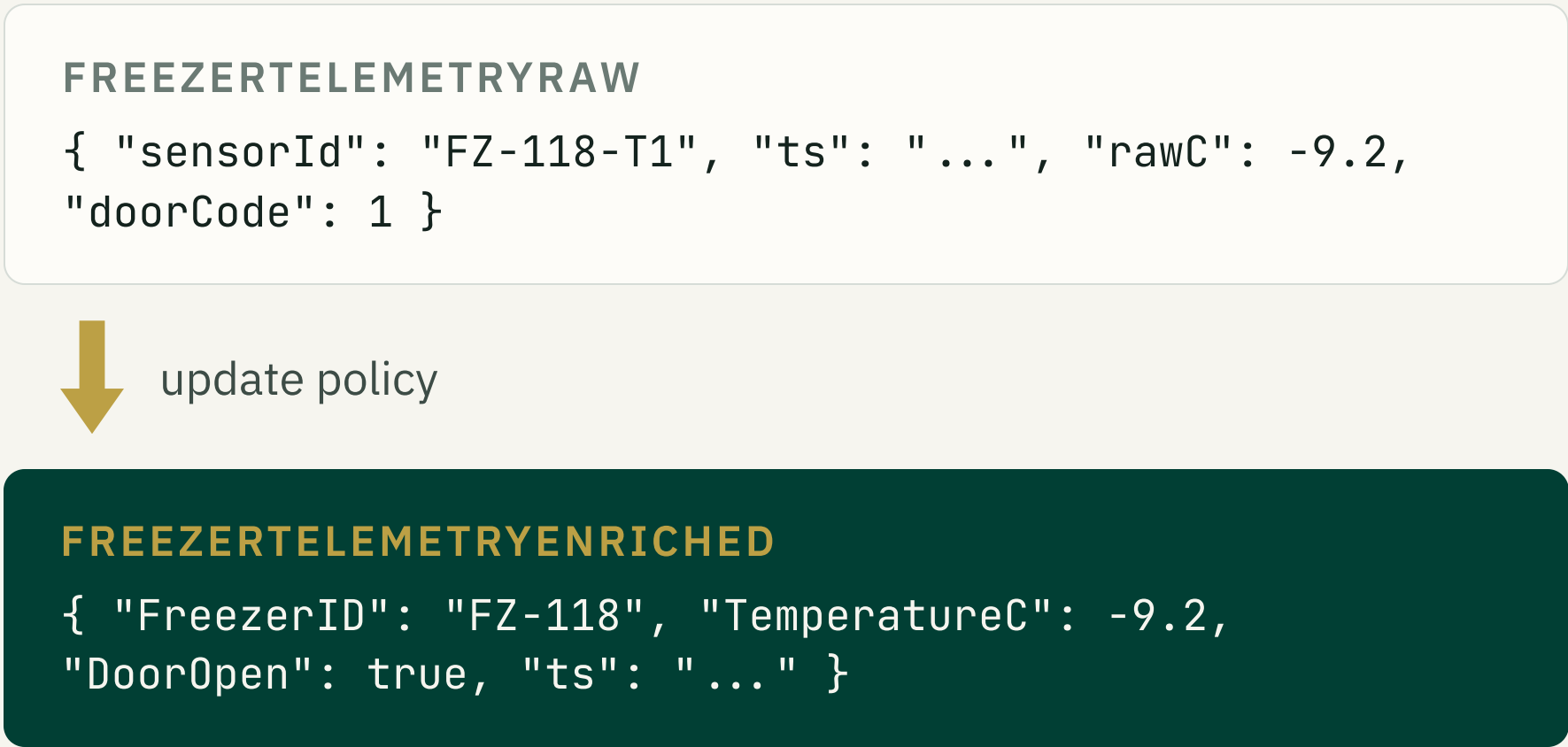


- Normalizes units, filters bad readings, computes DoorOpen from raw sensor codes
- A **materialized view** could maintain "latest reading per Freezer" for fast dashboard/agent queries
- Neither touches the raw stream — both are additive, non-destructive transforms



This is the plumbing. The ontology in Module 11 is what gives it business meaning.

Raw event to enriched record



RunPreviewRecallShare querySave to DashboardKQL ToolsExport to CSVMore...

```
1
2 FreezerTelemetryEnriched
3 | take 20
4
```

FreezerId	Timestamp	StoreId	TemperatureC	DoorOpen	StoreName	Region	City	Model
> FRZ001	2026-09-16 10:57:29.3844550	ST01	-17.88	false	Fabrikam Fresh Eixample	Catalonia	Barcelona	Liebherr GGPv 68
> FRZ002	2026-09-16 10:57:29.3844780	ST01	-18.19	false	Fabrikam Fresh Eixample	Catalonia	Barcelona	Carrier Viento 55
> FRZ003	2026-09-16 10:57:29.3844810	ST01	-18.39	false	Fabrikam Fresh Eixample	Catalonia	Barcelona	Epta Bering M2
> FRZ004	2026-09-16 10:57:29.3844830	ST02	-18.1	false	Fabrikam Fresh Gracia	Catalonia	Barcelona	Arneg Genesis C
> FRZ005	2026-09-16 10:57:29.3844850	ST02	-18.37	false	Fabrikam Fresh Gracia	Catalonia	Barcelona	Zoeller Iceberg 4
> FRZ006	2026-09-16 10:57:29.3844870	ST03	-17.99	false	Fabrikam Fresh Malasana	Madrid	Madrid	Liebherr GGPv 68
> FRZ007	2026-09-16 10:57:29.3844890	ST03	-17.4	false	Fabrikam Fresh Malasana	Madrid	Madrid	Framec Frare 80
> FRZ008	2026-09-16 10:57:29.3844910	ST03	-14.66	true	Fabrikam Fresh Malasana	Madrid	Madrid	Criocabin Frozen
> FRZ009	2026-09-16 10:57:29.3844930	ST04	-18.09	false	Fabrikam Fresh Ruzafa	Valencia	Valencia	Carrier Viento 55
> FRZ010	2026-09-16 10:57:29.3844950	ST04	-17.18	false	Fabrikam Fresh Ruzafa	Valencia	Valencia	Epta Bering M2
> FRZ011	2026-09-16 10:57:29.3844970	ST05	-17.75	false	Fabrikam Fresh Abando	Basque Country	Bilbao	Arneg Genesis C
> FRZ012	2026-09-16 10:57:29.3844990	ST05	-18	false	Fabrikam Fresh Abando	Basque Country	Bilbao	Zoeller Iceberg 4
> FRZ013	2026-09-16 10:57:29.3845010	ST05	-18.32	false	Fabrikam Fresh Abando	Basque Country	Bilbao	Framec Frare 80
> FRZ014	2026-09-16 10:57:29.3845020	ST06	-17.3	false	Fabrikam Fresh Triana	Andalusia	Seville	Liebherr GGPv 68
> FRZ015	2026-09-16 10:57:29.3845040	ST06	-17.83	false	Fabrikam Fresh Triana	Andalusia	Seville	Criocabin Frozen

FreezerTelemetryEnriched, keyed by FreezerId and already carrying StoreName, Region, City

Same event, now keyed by FreezerID — ready to be bound to a Freezer entity in Module 11.

TOPIC 4

Where this leaves us

- ✓ Static context landed in ColdChainLakehouse (Customers, Stores, Freezers)
- ✓ Live telemetry cleaned and keyed in ColdChainEventhouse (FreezerTelemetryEnriched)
- ✓ Both sides share consistent FreezerID / StoreID / CustomerID keys

▶ Next: bind them together in a queryable ontology graph

HANDS-ON LAB

Lab 10 — Telemetry & grounding

25 minutes

- Land reference CSVs into ColdChainLakehouse
- Wire the freezer telemetry generator into FreezerTelemetryEventstream
- Inspect the update policy that produces FreezerTelemetryEnriched



LAB GUIDE

aka.bi/lab10

MODULE 11 • TOPIC 5 — THE CORE OF THIS WORKSHOP

Ontology design essentials for operational AI

15 min theory • 30 min lab

BY THE END OF THIS MODULE YOU CAN

- Define entity type, property, and relationship in Fabric IQ's ontology model
- Explain how ontology binding works **without copying data**
- Describe why the ontology graph is queryable, not just descriptive
- Walk through the design of ColdChainOntology end to end

TOPIC 5

What is an ontology, here?

A **business-language model** of the things that matter: Customers, Stores, Freezers. It sits *above* the Lakehouse and Eventhouse tables from Module 10 — it doesn't replace them.

Entity type

A kind of thing

Freezer

Not a table — a business concept that may be **bound** to one or more tables

Property

A fact about that thing

TemperatureC, Model

Bound to a source column — the ontology stores the *mapping*, not the *value*

Relationship

How entity types connect

Store → has → Freezer

Named and directional — makes the graph **traversable**

Two flavors, both valid on the same entity type

Static

Rarely changes

`Freezer.Model` • `Freezer.Capacity` •
`Freezer.InstallDate`

Source: Lakehouse

Live

Changes continuously

`Freezer.TemperatureC` • `Freezer.DoorOpen`

Source: Eventhouse

Properties are bound to source columns — the ontology stores the *mapping*, not the *value*.

A named, directional connection between two entity types

Store → has → Freezer

Customer → shops at → Store

- Relationships make the graph **traversable**
- "Show me every Freezer at every Store this Customer shops at"

Add new relationship
Add additional semantic context and mirror real world relationships between entity types.

Relationship type name *

has

Origin entity type *

Store

Target entity type *

Freezer

Next: You'll refine the data definition for this new relationship and add an optional mapping table.

Create Cancel

Creating Store —has→ Freezer in the ontology designer

TOPIC 5

No-code binding, without copying data

- Binding an entity type's property to a table column is done **declaratively, in the ontology designer**
- No pipeline, no notebook, no data movement
- The ontology stores a **pointer**: "Freezer.Model comes from ColdChainLakehouse.Freezers.Model"
- Change the underlying data, and the ontology reflects it immediately — nothing to resync



This is the payoff of Module 10's work: because both sides already share clean keys, binding is just a few clicks.

TOPIC 5

The queryable ontology graph

- Once entities, properties, and relationships are defined, the ontology is a **live graph** — not documentation
- Query it directly: traverse relationships, filter on properties, mix static and live facts in one query
- This graph is what **agents** query in Module 12 — they never touch raw tables directly
- The same graph powers Copilot-style natural-language question answering

Home > Freezer

Manage rules

Configure Instances Overview

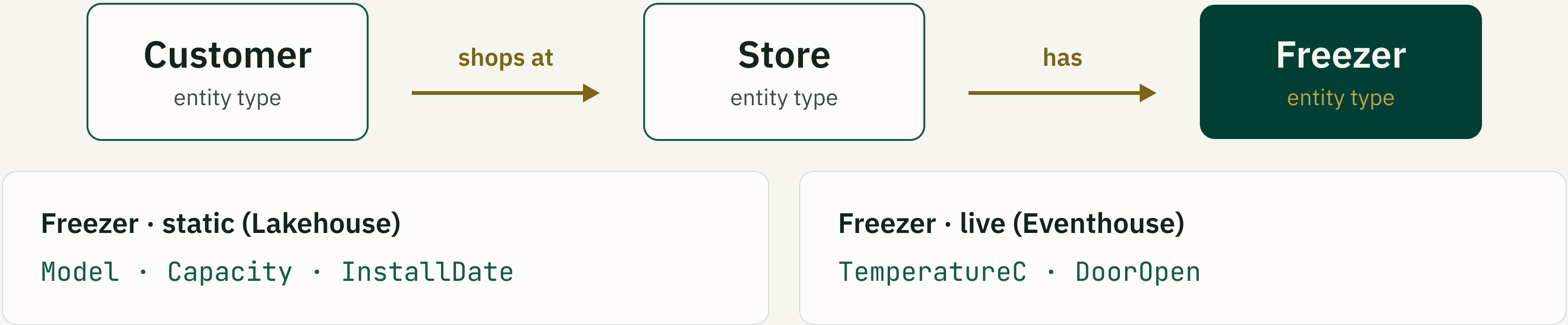
Count: 15

Filter by keyword

Capacity	FreezerId	InstallDate	Model	StoreId
650	FRZ001	2022-03-14T00:00:00	Liebherr GGPv 6570	ST01
550	FRZ002	2022-03-14T00:00:00	Carrier Viento 550	ST01
700	FRZ003	2023-06-01T00:00:00	Epta Bering M2	ST01
600	FRZ004	2021-11-08T00:00:00	Arneg Genesis Cube	ST02

Freezer instances resolved through the ontology

ColdChainOntology



This is the graph our agents will reason over in Module 12. Every node is traceable back to a real table — nothing here is duplicated data.

TOPIC 5

Freezer: a dual-bound entity type

Static binding

`ColdChainLakehouse.Freezers`

Model · Capacity · InstallDate

Live binding

`ColdChainEventhouse.FreezerTelemetryEnriched`

TemperatureC · DoorOpen



One entity, two speeds of truth. Nameplate specs and live telemetry, on the same node in the graph.

The screenshot shows the configuration for the 'Freezer' entity type. The 'Entity type key' is 'FreezerId'. Under 'Binding selection', the 'freezers' binding is selected. A dropdown menu for 'Add data binding' is open, showing 'Lakehouse table' and 'Eventhouse table or materialized view'. The 'Entity type key mapping' section shows 'FreezerId' mapped to 'FreezerId'. The 'Timeseries data' section is empty. The 'Properties' section is empty. 'Save' and 'Cancel' buttons are at the bottom.

Adding the second, Eventhouse binding to Freezer

TOPIC 5

Design principles worth internalizing

Model business concepts

Entity types around what the business talks about, not around whatever tables happen to exist

Name relationships

Directional and named — "has" and "shops at" read like a sentence, not a foreign key

Bind narrowly

Don't drag in every column just because it's there

Answer plain English

A good ontology answers *"which freezers does this customer's usual store have?"* by construction

HANDS-ON LAB

Lab 11 — Build ColdChainOntology

30 minutes

- Create entity types: Customer, Store, Freezer
- Bind Freezer to both ColdChainLakehouse.Freezers and ColdChainEventhouse.FreezerTelemetryEnriched
- Define relationships and run your first graph query



LAB GUIDE

aka.bi/lab11

MODULE 12 • TOPIC 6

Agent patterns over real-time and semantic layers

15 min theory • 20 min lab

BY THE END OF THIS MODULE YOU CAN

- Name four distinct ways to ground an agent in an ontology, and when to reach for each
- Explain what ColdChainDataAgent and ColdChainOperationsAgent each do
- Describe how an Activator Ontology Rule differs from a raw-threshold Activator rule

TOPIC 6

Why agents need grounding

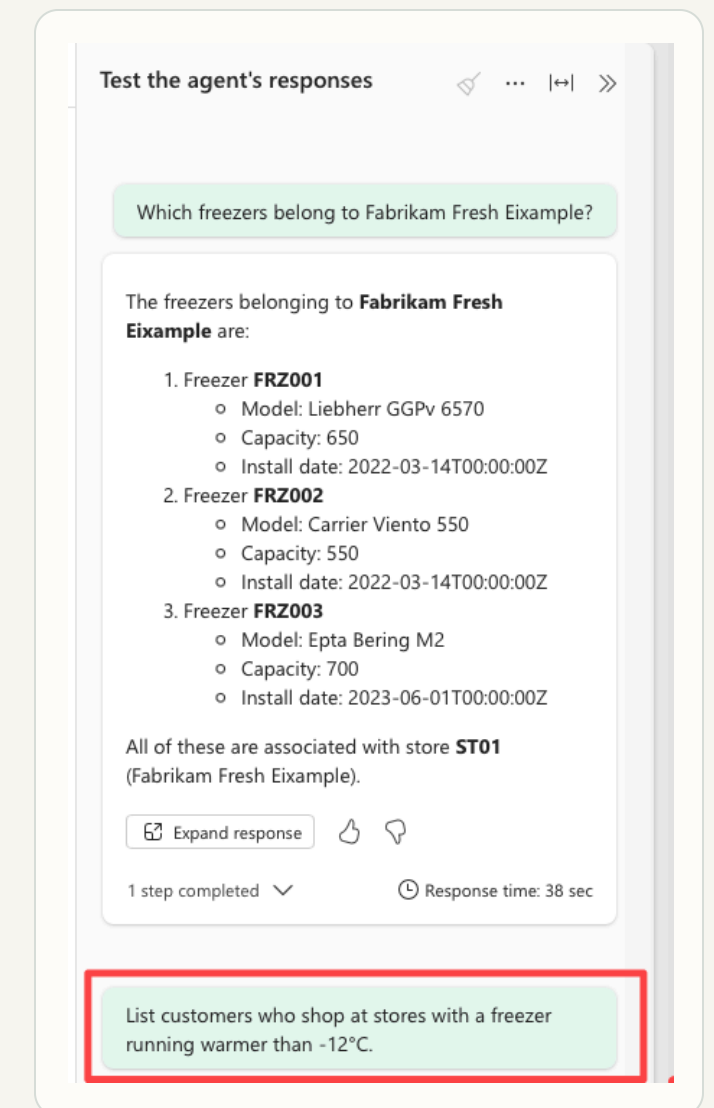
- An LLM with no context will guess, hallucinate units, or invent entities that don't exist
- ColdChainOntology gives an agent a **fixed vocabulary**: real entity types, real properties, real relationships
- Every question an agent answers, and every action it takes, can be traced back to a graph node
- This module is about **where** an agent plugs into that graph

Four ways to ground an agent

Pattern	Grounding style	Best for
Fabric Data Agent	Q&A over the ontology graph	Ask-a-question, exploratory analysis
Operations Agent	Event-triggered, paired with rules	React to conditions as they happen
Foundry IQ	Ontology as a tool for custom AI apps	Bespoke agentic apps in Azure AI Foundry
Copilot Studio MCP connector	Ontology exposed via MCP	Grounding agents built outside Fabric entirely

ColdChainDataAgent

- Grounded directly in **ColdChainOntology**
- Answers natural-language questions by traversing the graph
- *"Which freezers belong to Fabrikam Fresh Eixample?"* → walks Store —has→ Freezer
- Question-and-answer pattern — a person (or another system) initiates



No joins written — the agent walked the relationship

ColdChainOperationsAgent

- Also grounded in ColdChainOntology, but **event-triggered rather than question-driven**
- Paired with an **Activator Ontology Rule**: "Freezer running warm"
- Fires when TemperatureC rises above roughly **-12°C** for a sustained period
- Simulates a real operational fault: **door left open, or compressor failure**

The screenshot displays an 'Agent playbook' interface. At the top, it says 'Agent playbook' with an information icon. Below this is a notification bar: 'Created with AI. Mistakes are possible. Read terms.' with a close button. The main content is divided into two sections. The first section is titled 'Business term glossary' and explains that it shows business concepts inferred from goals, instructions, and data, along with tables and columns used to track data. It then defines the 'Class: Freezer' as 'A freezer monitored for temperature and store location context'. Under 'Properties', it lists: 'StoreName (string): Store name where the freezer is located', 'Region (string): Region where the freezer is located', 'FreezerId (string): Identifier of the freezer', 'TemperatureC (decimal): Current freezer temperature in Celsius', and 'City (string): City where the freezer is located'. The second section is titled 'Rules' and explains that these are rules the agent will follow based on its understanding of business goals and instructions. It contains a rule titled 'Freezer Temperature Above Threshold Alert: Alert on Teams whenever a freezer's temperatureC is above 12°C, including store location context.' with a star icon and a dropdown arrow.

The generated playbook speaks in entity terms, not table columns

TOPIC 6

Activator Ontology Rules vs. classic Activator rules

Classic Activator (RTI half)

Threshold on a raw column

"Alert if TemperatureC > -12"

No idea what entity, store, or customer is involved

Activator Ontology Rule

Threshold on an **entity's property**

*"Alert if this **Freezer** is running warm"*

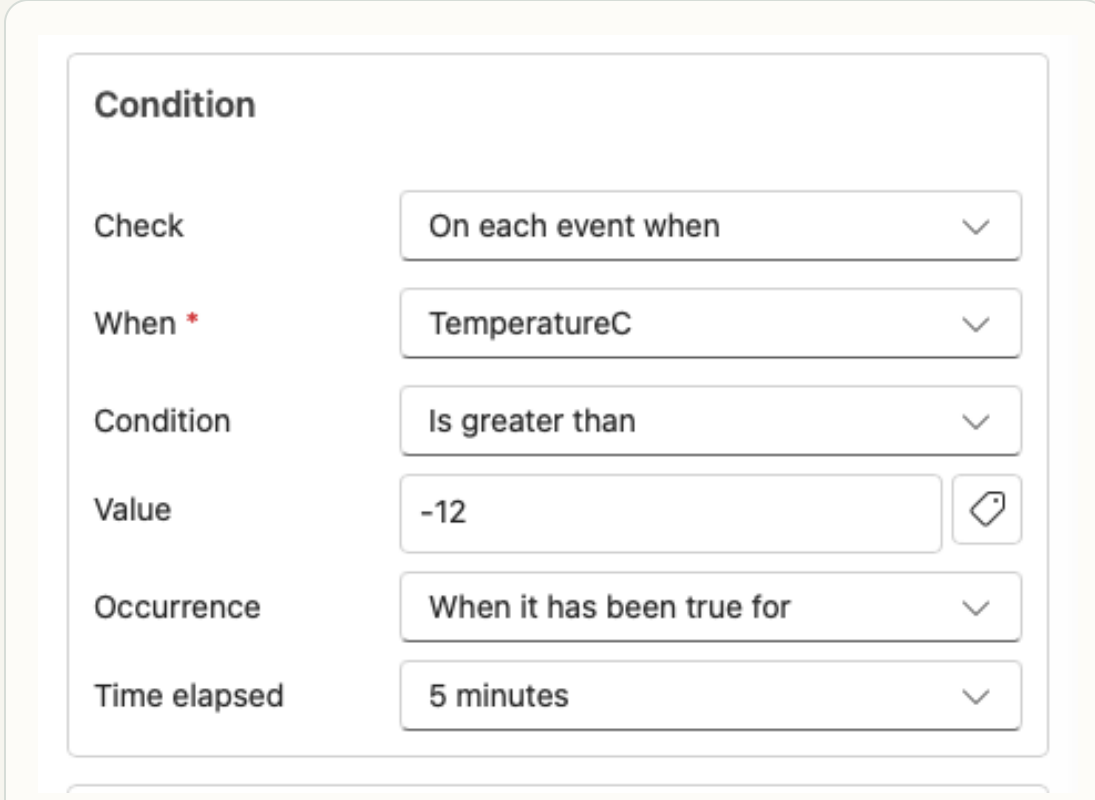
Carries the whole graph context: which Store, which Customer base



Same math, radically different output — one gives you a number, the other gives you a business incident.

"Freezer running warm" end to end

- 1 FreezerTelemetryEnriched.TemperatureC climbs above $\approx$ **-12°C** and stays there
- 2 The **Activator Ontology Rule** evaluates this against the Freezer entity, not a raw column
- 3 Rule fires → **ColdChainOperationsAgent** is notified
- 4 Agent has full graph context: *this* Freezer, *its* Store, *its* Model/Capacity/InstallDate
- 5 Agent acts or notifies **in business language**: *"Freezer FZ-118 at Nørrebro is running warm — check the door or compressor"*



The screenshot shows a configuration window for a rule condition. It contains several rows, each with a label on the left and a control on the right. The controls include dropdown menus and a text input field. The 'Value' row has a text input with '-12' and a small icon to its right. The 'Condition' row has a dropdown menu showing 'Is greater than'. The 'Occurrence' row has a dropdown menu showing 'When it has been true for'. The 'Time elapsed' row has a dropdown menu showing '5 minutes'. The 'Check' row has a dropdown menu showing 'On each event when'. The 'When *' row has a dropdown menu showing 'TemperatureC'.

Condition	
Check	On each event when
When *	TemperatureC
Condition	Is greater than
Value	-12
Occurrence	When it has been true for
Time elapsed	5 minutes

The rule: greater than -12, sustained for 5 minutes

Grounding beyond Fabric's built-in agents

Pattern 3 · Foundry IQ

Exposes the ontology as a grounding source for custom agentic applications built in **Azure AI Foundry**

Use when you need an agent experience **beyond** what Fabric's built-in Data/Operations Agents offer

Same underlying graph, same trust guarantees — different build surface

Pattern 4 · Copilot Studio MCP connector

Fabric IQ exposes the ontology via **MCP (Model Context Protocol)**

An agent built entirely **outside Fabric** — e.g. in Copilot Studio — grounds its answers in ColdChainOntology

The ontology becomes a reusable grounding source across your whole agent estate

TOPIC 6

Choosing a pattern

If you need...	Reach for
Ad-hoc Q&A over the graph	Data Agent
To react automatically when something changes	Operations Agent + Ontology Rule
A custom AI app in Foundry	Foundry IQ
To ground an agent that lives outside Fabric	Copilot Studio MCP connector



All four read from the **same ontology** — no duplicated modeling effort.

HANDS-ON LAB

Lab 12 — Agent patterns

20 minutes

- Build ColdChainDataAgent, grounded in ColdChainOntology
- Create the "**Freezer running warm**" Activator Ontology Rule
- Build ColdChainOperationsAgent and trigger it with live telemetry



LAB GUIDE

aka.bi/lab12

MODULE 13 • TOPICS 7–8

Prompting, trust & traceability

10 min theory • 5 min lab — the built-in buffer

BY THE END OF THIS MODULE YOU CAN

- Explain how ontology grounding constrains and validates an agent's responses
- Contrast a grounded agent's answer with a free-form LLM answer on the same question
- Describe what an audit trail looks like for an ontology-triggered agent action

TOPIC 7

Prompting, grounding, and response validation

- A prompt to ColdChainDataAgent isn't answered by the raw LLM alone
- The agent's response is **constrained to what the ontology graph actually contains**
- If a fact isn't a node, property, or relationship in ColdChainOntology, the agent can't fabricate it into an answer
- This is validation **by construction**, not a post-hoc check bolted on afterward

TOPIC 7

Grounded agent vs. free-form LLM

Free-form LLM

Answers from general training knowledge

No guaranteed link to *your* data

Can sound confident and still be wrong

No way to verify which record it "used"

Ontology-grounded agent

Answers by traversing real entities and properties

Every fact traces to a bound column

Wrong data → wrong answer, but never a *fabricated* entity

Response can cite the entity/property it used

TOPIC 7

Response validation strategies

Schema-constrained answers

The agent can only reference entity types/properties that exist in the ontology

Cite the source

A good response names the entity instance behind the answer (*"based on Freezer FZ-118..."*)

Confidence framing

Distinguish "the graph says X" from "I'm inferring X"

Fail closed

If the graph doesn't have the answer, the agent should say so, not guess

TOPIC 8

Trust, transparency, and traceability

An agent that can *act* (like ColdChainOperationsAgent) needs more than a good answer — it needs an **audit trail**. Three questions every stakeholder will ask after an incident:

What

triggered this action?

Which entity

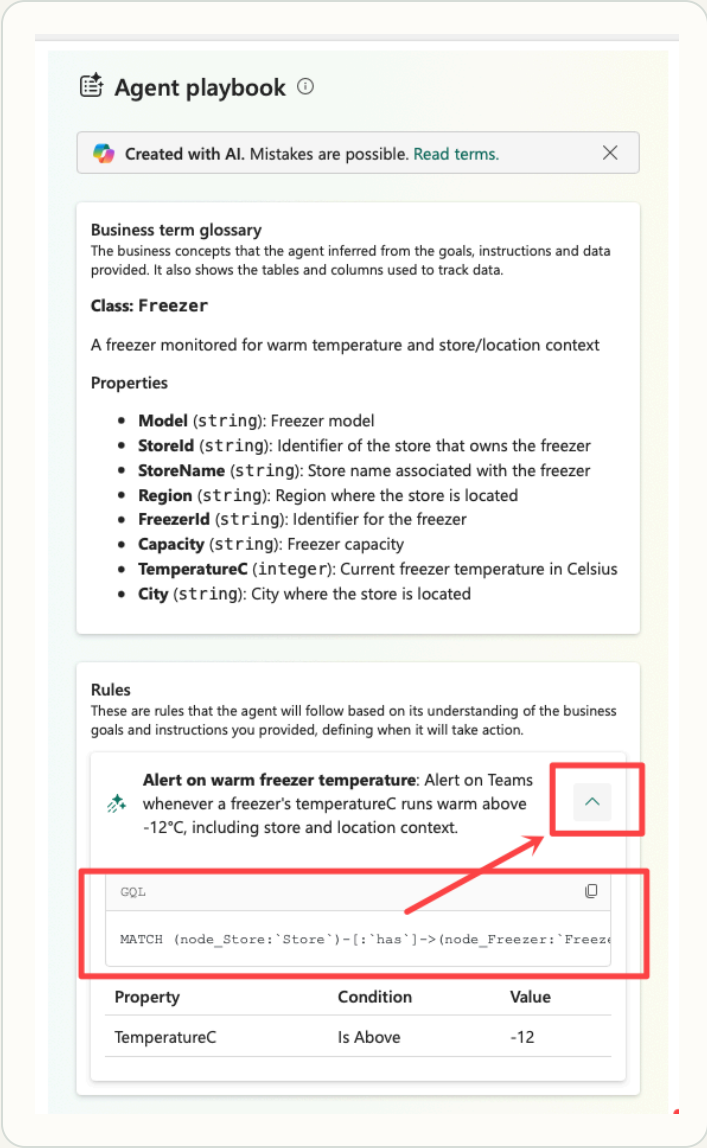
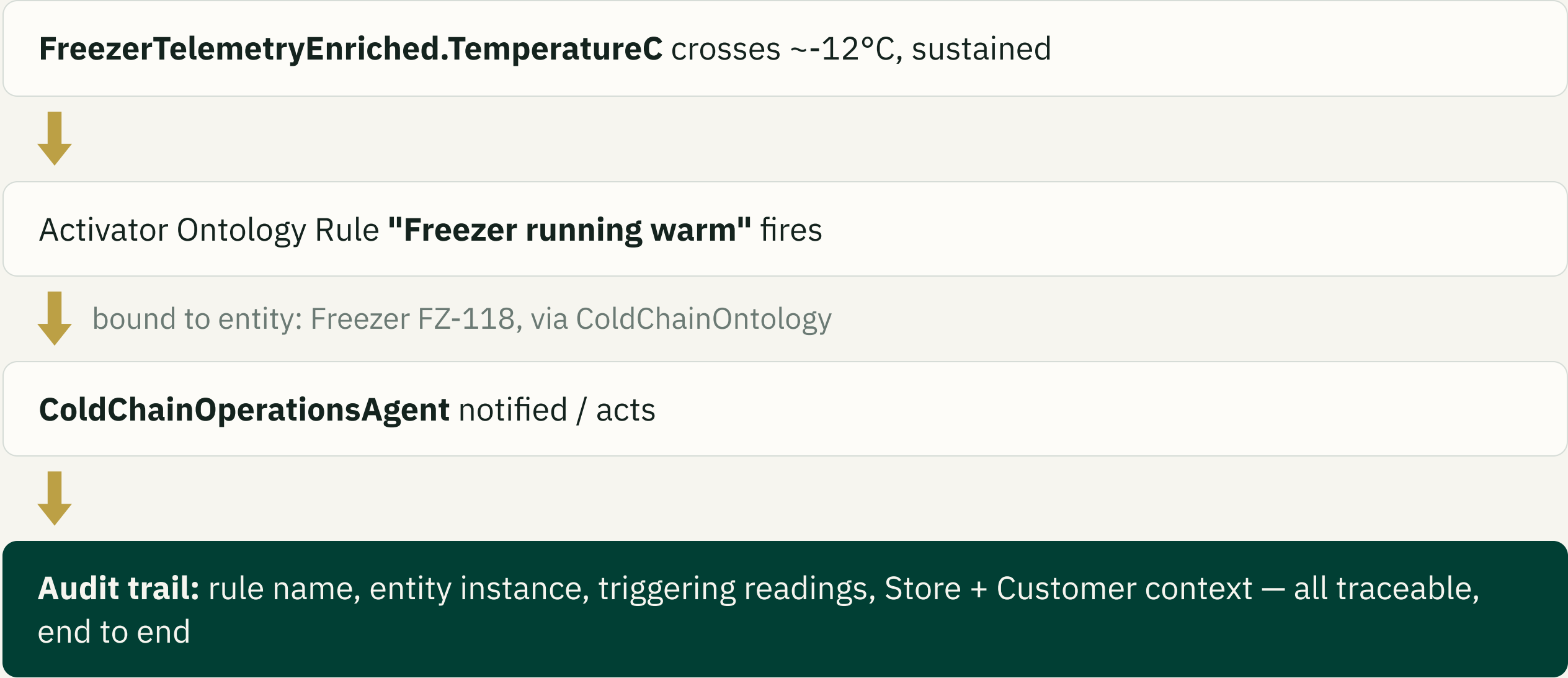
was it about?

What data

was the decision based on?

Ontology grounding makes all three answerable by design.

Tracing "Freezer running warm"



The rule's actual GQL condition, inspected

TOPIC 8

Why this matters operationally

- A manager asking "*why did this fire?*" gets a real answer, not a log line with a sensor ID
- Compliance/audit reviews can replay **which entity, which rule, which underlying reading** caused an action
- Trust compounds: once one traced incident checks out, the room trusts the next ten



This is the difference between an agent you **monitor** and one you **rely on**.

Bringing it together

Grounding (Topic 7)

prevents the agent from saying something false

Traceability (Topic 8)

lets you prove *why* it said or did something true

- Both come from the same source: the ontology graph built in Module 11
- Everything downstream — Data Agent answers, Operations Agent actions — inherits this for free

HANDS-ON LAB

Lab 13 — Prompting, trust & traceability

5 minutes

- Ask ColdChainDataAgent a question and inspect which entities/properties it cited
- Trace a fired "Freezer running warm" event back to its source reading



LAB GUIDE

aka.bi/lab13

MODULE 14

Wrap-up

10 min · recap, resources, next steps

BY THE END OF THIS MODULE YOU CAN

- Recap the arc of the day
- Take the resources home
- Clean up and give feedback

MODULE 14

What you built today

LAKEHOUSE

ColdChainLakehouse

Customers · Stores · Freezers

EVENTHOUSE

ColdChainEventhouse

ColdChainKQLDB: FreezerTelemetryRaw →
FreezerTelemetryEnriched

EVENTSTREAM

FreezerTelemetryEventstream

Live simulated freezer readings

ONTOLOGY

ColdChainOntology

Customer —shops at→ Store —has→
Freezer (static + live bound)

DATA AGENT

ColdChainDataAgent

Grounded in ColdChainOntology

OPERATIONS AGENT

ColdChainOperationsAgent

+ "Freezer running warm" rule

All inside one workspace: **Fabric IQ**

MODULE 14

The arc of the day

- 1 **Module 09** — a raw reading means nothing without context
- 2 **Module 10** — prepared static context and live telemetry to be groundable
- 3 **Module 11** — built the ontology that gives every reading a business meaning
- 4 **Module 12** — put agents on top that reason and act in business language
- 5 **Module 13** — saw how grounding makes those agents trustworthy and auditable



Same idea, four different layers: workspace → data → ontology → agents.

MODULE 14

Official Microsoft tutorials this workshop adapted

- [Get started with Fabric IQ](#)
- [Fabric IQ overview](#)
- [Ontology tutorial 0 — Introduction](#)
- [Ontology tutorial 1 — Create an ontology](#)
- [Ontology tutorial 4 — Create a data agent](#)
- [Digital Twin Builder RTI tutorial](#)
- [Kusto update policies](#)
- [Kusto materialized views — use cases](#)
- [Create an operations agent](#)
- [Ontology rules](#)
- [Agent integration concepts](#)
- [Fabric IQ Ontology MCP connector](#)
- [mslearn-fabric hands-on labs](#)

MODULE 14

Keep learning after today

Rerun the labs

In your own tenant — the modules/ folder is yours to keep

Extend the ontology

Add a new entity type (e.g. MaintenanceTicket) to ColdChainOntology as a self-study exercise

Ground an external agent

Try the **Copilot Studio MCP connector** against your ontology

Explore Foundry IQ

If you're already building custom agentic apps in Azure AI Foundry

MODULE 14

Before you go

- If you provisioned a trial/temporary capacity just for today, remember to **clean up** the Fabric IQ workspace to avoid ongoing charges
- Slides, lab guides, and setup scripts all live in this repo — clone it, don't just screenshot it
- Feedback helps us improve the next run of this workshop — please fill in the session survey

Thank you

Questions, feedback, and freezer-alert war stories always welcome.